

Waveform Examples

```

(
  NF(\iop, {|freq=78, mul=1.0, add=0.0|
    var noise = LFNoise1.ar(0.001).range(freq, freq + (freq * 0.1));
    var osc = SinOsc.ar([noise, noise * 1.04, noise * 1.02, noise * 1.08],0,0.2);
    var out = DFm1.ar(osc,freq*4,SinOsc.kr(0.01).range(0.92,1.05),1,0,0.005,0.7);
    HPF.ar(out, 40)
  }).play;
)

(
  NF(\dsc, {|freq = 1080|
    HPF.ar(
      BBandStop.ar(Saw.ar(LFNoise1.ar([19,12]).range(freq,freq*2), 0.2).excess(
        SinOsc.ar( [freq + 6, freq + 4, freq + 2, freq + 8])),
        LFNoise1.ar([12,14,10]).range(100,900),
        SinOsc.ar(20).range(9,11)
      ), 80)
    ).play;
)

var <>pindex, <>cindex;

initialize {
  if(pindex.isNil, { pindex = 1000 });
  if(cindex.isNil, { cindex = 2000 });
}

clearProcessSlots {
  pindex = 1000;
  (this.pindex - 1000).do{|i| this[this.pindex+i] = nil; }
}

clearOrInit {|clear=true|
  if(clear == true, { this.clearProcessSlots() }, { this.initialize() });
}

transform {|process, index|
  if(index.isNil && pindex.isNil, {
    this.initialize();
  });

  pindex = pindex + 1;
  this[pindex] = \filter -> process;
}

control {|process, index|
  var i = index;

  if(i.isNil, {
    this.initialize();
    cindex = cindex + 1;
    i = cindex;
  });

  this[i] = \pset -> process;
}

(
  NF(\depfm, {|freqMin=5, freqMax=20, mul=20, add=80, rate=0.5, modFreq=2100, index=0.3, amp=0.2|
    var trig, seq, freq;
    trig = Dust.kr(rate);
    seq = Diwhite(freqMin, freqMax, inf).midicps;
    freq = Demand.kr(trig, 0, seq);
    HPF.ar(PMOsc.ar(LFCub.kr([freq, freq/2, freq/3, freq/4], 0, mul, add),
      LFNoise1.ar(0.3).range(modFreq,modFreq*2), index) * amp, 50)
  }).play;
)

```

Exercises

Exercises

1. Implement a **pattern process** for waveset distortion that uses different patterns to control the **play rate** and **repeats** parameters.
2. Implement a **pattern process** for waveset distortion that adds a **per-waveset transformation** such as a filter, glissandi or binary transformation.
3. Create a **DemandEnvGen process** that contains at least two layers of the main DemandEnvGen Ugen where the envelope points are created using **Dbrown** or **Dbrown2**
4. Create a **DemandEnvGen process** that forms a varying triangle waveform.
5. Combine the **binary operations** so that different ones are used for the high frequencies of the input sound and different ones for the low frequencies.

Exercises (from last time)

1. Create a **demand rate waveform** that contains two different demand patterns for creating both durations and levels.
2. Implement a SynthDef with at least two or more **Gendy oscillators**. These should additionally be cross-modulated using binary operations.
3. **Generate a buffer** to read with Osc based on at least three different ways of creating random numbers.
4. Implement a pattern for reading a sample that has been **analysed to wavesets**. The pattern should brownian motion for its movements.
5. Experiment by **creating a shape** that will be used at the same time as a waveform for synthesis and an envelope for a sound object.

